

Mission Control: Data Center Rebuild

Warning: This procedure removes the existing failed data center, including **deletion of all resources and data** in the failed data center. The active (surviving) data center continues to operate normally. Do not run this procedure in a production environment without fully understanding and accepting these impacts. It must only be run when the data in the failed data center will never need to be recovered, for example because it is corrupted, lost, or out-of-date.

Overview

This section describes how to rebuild a failed Usage database data center in a multi-datacenter DSE cluster. The procedure is intended for outages longer than hint window size. At a high level, the failed datacenter needs to be removed first from the cluster. This leads to **deletion of all resources and data** in the failed data center; the active data center continues to operate normally. After that, the data center has to be added back to the cluster and the data needs to be streamed to new nodes from the surviving data center. In some cases, such as insufficient disk space, failed cluster update, or any other situation leading to the data center not being in ready state, the `K8ssandraOperator` may refuse to decommission the data center. In these scenarios, the data center must be forcefully removed. The procedure is described in the last section of this document.

The example shown uses a two-datacenter cluster with `east` (surviving) and `west` (failed). Each data center is configured with three racks and 9 nodes (3 nodes per rack), with a replication factor of 3 per data center.

Prerequisites

- `kubectl` access allowing modification removal and creation of the resources in the database keyspace
- Surviving datacenter must be in consistent state
- `CQLSH` access with ALTER keyspace permission

Rebuild Procedure

The rebuild procedure consists of the following steps

- Setting replication to surviving datacenter only
- Decommissioning failed datacenter
- Optionally forcefully removing failed datacenter
- Adding back data-center
- Repairing system keyspaces on the surviving data-center
- Restoring replication to both data-centers
- Rebuilding new data-center from surviving data-center

Note: The Mission Control UI does not provide an option to rebuild a data center, so this manual MOP must be followed.

Remove Replication to the Failed Data Center

The first step in the procedure is to reconfigure all user keyspaces to replicate to the surviving data center only. Login with `cqlsh` to one of the nodes in the surviving data center and alter all user defined keyspaces as follows. In the example below two user keyspaces exist `ads` and `reporting`.

```
db-admin@cqlsh> ALTER KEYSPACE ads WITH replication = {'class': 'NetworkTopology'}
Warnings :
When increasing replication factor you need to run a non-incremental repair
db-admin@cqlsh> ALTER KEYSPACE reporting WITH replication = {'class': 'NetworkTopology'}
Warnings :
When increasing replication factor you need to run a non-incremental repair
```

The warning from the `ALTER KEYSPACE` can be ignored at this stage as the replication factor is not changed in the surviving data center.

Remove the failed data-center from `MissionControlCluster`

Remove the item from the `.spec.k8ssandra.cassandra.datacenters` list describing the failed datacenter in the `MissionControlCluster` object and apply it. The section in question is shown below:

```

- datacenterName: west
  metadata:
    name: west
  racks:
  - name: rack1
    nodeAffinityLabels:
      mission-control.datastax.com/role: database
      topology.kubernetes.io/zone: northeurope-1
  - name: rack2
    nodeAffinityLabels:
      mission-control.datastax.com/role: database
      topology.kubernetes.io/zone: northeurope-2
  - name: rack3
    nodeAffinityLabels:
      mission-control.datastax.com/role: database
      topology.kubernetes.io/zone: northeurope-3
  size: 9
  stopped: false

```

The above change should trigger:

- Removal of the CassandraDatacenter in the `west` Kubernetes cluster
- Removal of all `statefulsets` representing racks in the `west` data-center
- Removal of all data `pvc`s in the `west` data-center

To verify the above is complete execute following command in the `west` Kubernetes cluster.

```

[k8s:aks-ads-west]$ kubectl get CassandraDatacenter,pvc,sts,pod -n ads-db
No resources found in ads-db namespace.

```

The status in the surviving data center should show a single data center with all nodes in

`Up/Normal` state.

```
dse@db-east-rack1-sts-0:~$ nodetool status
```

```
Datacenter: east
```

```
=====
```

```
Status=Up/Down
```

```
|/ State=Normal/Leaving/Joining/Moving/Stopped
```

--	Address	Load	Tokens	Owns (effective)	Host ID
UN	10.1.8.49	10.85 TiB	16	34.3%	b2514502-1d40-469
UN	10.1.8.164	10.61 TiB	16	33.3%	e727d534-dade-4ad
UN	10.1.8.71	10.2 TiB	16	32.3%	e0eb6992-2503-4fc
UN	10.1.8.233	10.5 TiB	16	32.3%	d7b1f2e8-f24a-4ad
UN	10.1.8.173	10.51 TiB	16	32.3%	266a3e9e-7503-43c
UN	10.1.8.204	11.15 TiB	16	34.3%	f95196be-24c5-4c8
UN	10.1.8.95	11.12 TiB	16	34.3%	10e1b061-ba2d-44c
UN	10.1.8.191	10.86 TiB	16	33.3%	fe961ad3-724b-44f
UN	10.1.8.127	10.86 TiB	16	33.3%	151dde4f-0b61-4cc

At this stage the data is stored in the surviving data center only and all data in the failed datacenter has been removed. If the `CassandraDatacenter` is not removed follow the [forceful removal procedure](#).

Re-add the data-center

Add the data center definition back to the `MissionControlCluster` manifest and apply it. The section in question is identical to the original definition and is shown below:

```
- datacenterName: west
  metadata:
    name: west
  racks:
    - name: rack1
      nodeAffinityLabels:
        mission-control.datastax.com/role: database
        topology.kubernetes.io/zone: northeurope-1
    - name: rack2
      nodeAffinityLabels:
        mission-control.datastax.com/role: database
        topology.kubernetes.io/zone: northeurope-2
    - name: rack3
      nodeAffinityLabels:
        mission-control.datastax.com/role: database
        topology.kubernetes.io/zone: northeurope-3
  size: 9
  stopped: false
```

This re-creates the `CassandraDatacenter`, the stateful sets, the PVCs, and the pods. In the example environment, all 9 pods reached `Running` state in approximately 16 minutes. To check completion of the data center creation execute the following command:

```
[k8s:aks-ads-west]$ kubectl get CassandraDatacenter,pvc,sts,pod -n ads-db
```

NAME	AGE
cassandradatacenter.cassandra.datastax.com/west	16m

NAME	STATUS	VOLUME
persistentvolumeclaim/server-data-db-west-rack1-sts-0	Bound	pvc-99a657
persistentvolumeclaim/server-data-db-west-rack1-sts-1	Bound	pvc-251f8b
persistentvolumeclaim/server-data-db-west-rack1-sts-2	Bound	pvc-f1a437
persistentvolumeclaim/server-data-db-west-rack2-sts-0	Bound	pvc-cef737
persistentvolumeclaim/server-data-db-west-rack2-sts-1	Bound	pvc-5280d6
persistentvolumeclaim/server-data-db-west-rack2-sts-2	Bound	pvc-81795e
persistentvolumeclaim/server-data-db-west-rack3-sts-0	Bound	pvc-1734cf
persistentvolumeclaim/server-data-db-west-rack3-sts-1	Bound	pvc-9fa9b2
persistentvolumeclaim/server-data-db-west-rack3-sts-2	Bound	pvc-a9634d

NAME	READY	AGE
statefulset.apps/db-west-rack1-sts	3/3	16m
statefulset.apps/db-west-rack2-sts	3/3	16m
statefulset.apps/db-west-rack3-sts	3/3	16m
statefulset.apps/db-west-reaper	0/0	16m

NAME	READY	STATUS	RESTARTS	AGE
pod/db-west-rack1-sts-0	2/2	Running	0	16m
pod/db-west-rack1-sts-1	2/2	Running	0	16m
pod/db-west-rack1-sts-2	2/2	Running	0	16m
pod/db-west-rack2-sts-0	2/2	Running	0	16m
pod/db-west-rack2-sts-1	2/2	Running	0	16m
pod/db-west-rack2-sts-2	2/2	Running	0	16m
pod/db-west-rack3-sts-0	2/2	Running	0	16m
pod/db-west-rack3-sts-1	2/2	Running	0	16m
pod/db-west-rack3-sts-2	2/2	Running	0	16m

`nodetool status ads` from the surviving data center reports all new nodes as `UN` (Up/Normal) but holding only a few hundred KiB of data. The keyspace replication is not yet aware of the new data center, as indicated by the warning at the end of the output:

```

Datacenter: east
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving/Stopped
-- Address      Load          Tokens      Owns (effective)  Host ID
UN  10.1.8.49     10.85 TiB    16          34.3%            b2514502-1d40-469
UN  10.1.8.164    10.61 TiB    16          33.3%            e727d534-dade-4ad
UN  10.1.8.71     10.2 TiB     16          32.3%            e0eb6992-2503-4fc
UN  10.1.8.233    10.5 TiB     16          32.3%            d7b1f2e8-f24a-4ad
UN  10.1.8.173    10.51 TiB    16          32.3%            266a3e9e-7503-43c
UN  10.1.8.204    11.15 TiB    16          34.3%            f95196be-24c5-4c8
UN  10.1.8.95     11.12 TiB    16          34.3%            10e1b061-ba2d-44c
UN  10.1.8.191    10.86 TiB    16          33.3%            fe961ad3-724b-44f
UN  10.1.8.127    10.86 TiB    16          33.3%            151dde4f-0b61-4cc

Datacenter: west
=====
-- Address      Load          Tokens      Owns      Host ID
UN  10.3.8.147    201.8 KiB     16         ?         72326989-6976-4f5a-8f2a-618c132
UN  10.3.8.121    179.9 KiB     16         ?         7af9134d-97d7-4e5b-b776-bc55adc
UN  10.3.8.17     134.8 KiB     16         ?         f92e12ee-ffe1-4c4c-80af-355d4f8
UN  10.3.8.21     118.1 KiB     16         ?         b64bfe42-01e3-472e-a229-41a1588
UN  10.3.8.211    121.1 KiB     16         ?         d16397ea-1e9c-4fd6-b20c-4a48446
UN  10.3.8.11     193.8 KiB     16         ?         638751cd-e139-48a6-b0a0-0822e20
UN  10.3.8.27     187.3 KiB     16         ?         e13902dd-d005-4777-a589-bad71ac
UN  10.3.8.39     195.4 KiB     16         ?         ef6dbc1b-04eb-4895-b891-f6f1b69
UN  10.3.8.87     211.7 KiB     16         ?         672f3a12-3a1b-4a8a-b3e4-68e0347
...

Note: Non-system keyspaces don't have the same replication settings. Please

```

Repair System Keyspaces

Before the traffic can be replicated to the new data-center, several system keyspaces, most importantly `dse_security` for the TDE encryption keys and `system_auth` for users and roles, must be repaired to ensure consistency. Run a primary-range repair on every node in the surviving data-center for each of these keyspaces:

```

for ks in system_auth dse_leases system_distributed dse_security dse_system
do
    for pod in $(kubectl get pods -n ads-db -l cassandra.datastax.com/datac
    do
        kubectl exec -it -n ads-db $pod -c cassandra -- nodetool repair -pr
    done
done
done

```

The duration of the system-keyspace repair scales with the number of nodes in the surviving data center. In the 9-node test cluster it took approximately 15 minutes.

Restore Replication to the Rebuilt Data Center

Restore the replication factor for the user keyspaces so that data will be streamed into the rebuilt data center:

```

db-admin@cqlsh> ALTER KEYSPACE ads WITH replication = {'class': 'NetworkTop

Warnings :
When increasing replication factor you need to run a non-incremental repair
db-admin@cqlsh> ALTER KEYSPACE reporting WITH replication = {'class': 'Netw

Warnings :
When increasing replication factor you need to run a non-incremental repair

```

The warning about running a non-incremental repair is expected and is addressed by the rebuild step that follows.

Once the replication factor includes the rebuilt data center, new write transactions begin flowing into it from the active data center. Progress can be monitored from the **Mission Control Cluster** dashboard, in the **Write Requests / Table** panel, where writes targeting the rebuilt data center should start appearing.



Start the Rebuild

Start the data rebuild by creating a `K8ssandraTask` that streams the `ads` keyspace from the surviving data center. The `K8ssandraTask` must be created in the control-plane data center, where the `K8ssandraOperator` is actively running. The configuration below rebuilds

the failed `west` data-center from the surviving `east` data-center (`source_datacenter: east`, `datacenters: [west]`). In this example we use `maxConcurrentPods: 2`. This means that up to two nodes in a rack will be rebuilt at the same time. The value of `maxConcurrentPods` should be calculated as `ceil(rackSize / 2)` to rebuild each rack in two steps. Rebuilding more than one rack at a time better utilizes available outbound streaming throughput in source nodes towards the end of rebuild. Unused bandwidth by one rebuilding node can be utilized by another.

Note: `maxConcurrentPods` is only available in Mission Control 1.18 and later. On earlier versions the rebuild task proceeds one node at a time.

```
apiVersion: control.k8ssandra.io/v1alpha1
kind: K8ssandraTask
metadata:
  name: west-data-rebuild-task
  namespace: ads-db
spec:
  cluster:
    name: db
    namespace: ads-db
  dcConcurrencyPolicy: Forbid
  datacenters:
  - west
  template:
    concurrencyPolicy: Allow
    maxConcurrentPods: 2
    jobs:
    - args:
        source_datacenter: east
        keyspace_name: ads
        command: rebuild
        name: rebuild-keyspace
      restartPolicy: Never
```

Monitor Rebuild Progress

The progress at the data center level can be checked by inspecting the relevant `K8ssandraTask`. It gives the high level view of completed nodes and the nodes in progress. The output shows just the status for brevity.

```
[k8s:aks-ads-east]$ kubectl describe K8ssandraTask -n ads-db west-data-rebu
...
```

Status:

Active: 1

Conditions:

Last Transition Time: 2026-04-22T06:37:49Z

Message:

Reason: Running

Status: True

Type: Running

Last Transition Time: 2026-04-22T06:37:49Z

Message:

Reason: Failed

Status: False

Type: Failed

Datacenters:

west:

Active: 1

Conditions:

Last Transition Time: 2026-04-22T06:37:49Z

Message:

Reason: Running

Status: True

Type: Running

Pod Statuses:

db-west-rack1-sts-0:

Completion Time: 2026-04-23T12:20:07Z

Job Id: c3fde597-5445-4776-ac55-d7c501b4fc2c

Start Time: 2026-04-22T06:37:49Z

Status: COMPLETED

db-west-rack1-sts-1:

Completion Time: 2026-04-23T12:20:07Z

Job Id: 28b2bfa1-bc48-4112-bef1-3926943b552b

Start Time: 2026-04-22T06:37:49Z

Status: COMPLETED

db-west-rack1-sts-2:

Completion Time: 2026-04-23T12:20:07Z

Job Id: 4caed229-ff02-4c04-9270-8737cb8deea8

Start Time: 2026-04-22T06:37:49Z

Status: COMPLETED

db-west-rack2-sts-0:

Job Id: 7887e1ce-0281-4cb2-8bcf-2c33c96fd09a

Start Time: 2026-04-23T12:20:07Z

Status: RUNNING

db-west-rack2-sts-1:

Job Id: 244f59f5-977f-497f-8528-9c094dffc001

Start Time: 2026-04-23T12:20:07Z

```
Status:      RUNNING
Start Time:  2026-04-22T06:37:49Z
Succeeded:   3
Start Time:  2026-04-22T06:37:49Z
Succeeded:   3
Events:      <none>
```

The progress of the rebuild in each node can be checked with the `nodetool netstats` command as below. It reports progress for each source endpoint separately giving percentage and volume of already streamed and total data.

```
dse@db-west-rack1-sts-0:~$ nodetool netstats | awk '/\s+\/([0-9]{1,3}\.){3}/'

/10.1.8.127 : 0.0265546%      0.301309/1134.68GB
/10.1.8.164 : 0.00364573%    0.046889/1286.14GB
/10.1.8.173 : 0.0159373%     0.179635/1127.14GB
/10.1.8.191 : 0.0343425%     0.360817/1050.64GB
/10.1.8.204 : 0.0165495%     0.190338/1150.11GB
/10.1.8.233 : 0.02645%       0.204172/771.92GB
/10.1.8.49  : 0.0021786%     0.0343957/1578.79GB
/10.1.8.71  : 0.00238363%    0.032081/1345.89GB
/10.1.8.95  : 0.0161406%     0.263193/1630.63GB
```

During a rebuild, the streaming traffic is visible on the network throughput of the receiving node. From the source side, data is streamed in parallel from multiple nodes in the surviving data center.

Forceful removal of the failed data center

Assassinate Failed Data Center Nodes

Forcefully remove every node from the failed data center from the cluster gossip state by running `nodetool assassinate` from the surviving data center against each of the failed node IP addresses. The IP addresses may be obtained from `nodetool status` command:

```
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.195
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.9.1
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.112
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.179
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.243
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.212
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.151
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.8.230
dse@db-west-rack1-sts-0:~$ nodetool assassinate 10.3.9.24
```

After all nodes have been assassinated, the cluster status reports only the surviving data center:

```
nodetool status

Datacenter: east
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving/Stopped
-- Address      Load          Tokens         Owns (effective)  Host ID
UN  10.1.8.49     10.9 TiB      16             34.3%             b2514502-1d40-469
UN  10.1.8.164    10.65 TiB     16             33.3%             e727d534-dade-4ad
UN  10.1.8.71     10.24 TiB     16             32.3%             e0eb6992-2503-4fc
UN  10.1.8.233    10.54 TiB     16             32.3%             d7b1f2e8-f24a-4ad
UN  10.1.8.173    10.55 TiB     16             32.3%             266a3e9e-7503-43c
UN  10.1.8.204    11.2 TiB      16             34.3%             f95196be-24c5-4c8
UN  10.1.8.95     11.16 TiB     16             34.3%             10e1b061-ba2d-44c
UN  10.1.8.191    10.9 TiB      16             33.3%             fe961ad3-724b-44f
UN  10.1.8.127    10.9 TiB      16             33.3%             151dde4f-0b61-4cc
```

Manually Delete the `CassandraDatacenter`

In normal operation, removing the data center from the `MissionControlCluster` propagates through the K8ssandra Cluster down to the `CassandraDatacenter` resource, and the Cass Operator removes the underlying pods, persistent volume claims (PVCs), and stateful sets.

In this case, because a node `K8ssandraOperator` is unable to decommission the data center, the `CassandraDatacenter` resource had to be deleted manually:

```
kubectl delete cassandradatacenter west -n ads-db
```

After manual deletion, the PVCs, pods, and stateful sets for the failed data center will be

removed. Make sure all resources are deleted before proceeding to next steps.